Industrial Internship Report on

"Multi-Client Service Platform Website"

Prepared by-
Tanushree Priyadarshini Sahoo

Executive Summary

This report provides details of the Industrial Internship provided by upskill Campus and The IoT Academy in collaboration with Industrial Partner UniConverge Technologies Pvt Ltd (UCT).
This internship was focused on a project/problem statement provided by UCT. We had to finish the project including the report in 6 weeks' time.
My project was the development of a **Multi-Client Service Platform Website**, a full-stack web application designed to seamlessly connect service providers (merchants) with customers. The platform features role-based dashboards, a secure cloud-based relational database architecture using MySQL hosted on TiDB cloud, and a robust Node.js backend (server.js). It integrates third-party functionalities including secure payment processing via Stripe and an automated email notification system, with the entire application deployed and hosted on Render.This internship gave me a very good opportunity to get exposure to Industrial problems and

design/implement solutions for that. It was an overall great experience to have this internship.

TABLE OF CONTENTS

1 Preface

This report summarizes six weeks of intensive full-stack development work, transitioning a multi-client platform from a basic structural framework to a functional, production-ready environment. Relevant internships are vital in bridging the gap between academic programming and industry-standard cloud architecture, particularly concerning system security, third-party API integration, and live deployment.
My project focused on engineering a robust Multi-Client Service Platform capable of handling asynchronous transactions and role-based data routing. The program was carefully planned: initial weeks focused on setting up a Node.js backend environment (managing dependencies via node modules) and designing the database. This was followed by migrating to a cloud-based MySQL database using TiDB cloud and managing it via MySQL Workbench. The latter weeks involved the complex integration of the Stripe payment gateway and automated email systems, concluding with live hosting deployment on Render and rigorous end-to-end testing. I gained tremendous practical knowledge in managing webhooks, securing API credentials in .env files, and ensuring seamless communication between a Node.js server and cloud databases. I extend my sincere gratitude to my mentors at UCT and upskill Campus for their guidance. To my juniors and peers: prioritize understanding backend security, modern cloud hosting platforms, and asynchronous data flows, as these are the cornerstones of scalable web services.

2 Introduction

2.1 About UniConverge Technologies Pvt Ltd

A company established in 2013 and working in Digital Transformation domain and providing Industrial solutions with prime focus on sustainability and RoI. For developing its products and solutions it is leveraging various Cutting Edge Technologies e.g. Internet of Things (IoT), Cyber Security, Cloud computing (AWS, Azure), Machine Learning, Communication Technologies (4G/5G/LoRaWAN), Java Full Stack, Python, Front end etc.

i. UCT IoT Platform (UCT Insight)

UCT Insight is an IOT platform designed for quick deployment of IOT applications on the same time providing valuable “insight” for your process/business. It has been built in Java for backend and ReactJS for Front end. It has support for MySQL and various NoSql Databases.

- It enables device connectivity via industry standard IoT protocols - MQTT, CoAP, HTTP, Modbus TCP, OPC UA
- It supports both cloud and on-premises deployments.

It has features to:

- Build Your own dashboard
- Analytics and Reporting
- Alert and Notification
- Integration with third party application(Power BI, SAP, ERP)
- Rule Engine

ii. Smart Factory Platform (Factory watch)

Factory watch is a platform for smart factory needs.
It provides Users/ Factory:

- with a scalable solution for their Production and asset monitoring
- OEE and predictive maintenance solution scaling up to digital twin for your assets.
- to unleased the true potential of the data that their machines are generating and helps to identify the KPIs and also improve them.
- A modular architecture that allows users to choose the service that they what to start and then can scale to more complex solutions as per their demands.

Its unique SaaS model helps users to save time, cost and money.

iii. LoRAWAN based Solution

UCT is one of the early adopters of LoRAWAN technology and providing solutions in Agritech, Smart cities, Industrial Monitoring, Smart Street Light, Smart Water/ Gas/ Electricity metering solutions etc.

iv. Predictive Maintenance

UCT is providing Industrial Machine health monitoring and Predictive maintenance solution leveraging Embedded system, Industrial IoT and Machine Learning Technologies by finding Remaining useful life time of various Machines used in production process.

2.2 About upskill Campus (USC)

upskill Campus along with The IoT Academy and in association with Uniconverge technologies has facilitated the smooth execution of the complete internship process.

USC is a career development platform that delivers personalized executive coaching in a more affordable, scalable and measurable way.
Seeing need of upskilling in self-paced manner along-with additional support services e.g. Internship, projects, interaction with Industry experts, Career growth Services.
https://www.upskillcampus.com/
upSkill Campus aiming to upskill 1 million learners in next 5 year.

2.3 The IoT Academy

The IoT academy is EdTech Division of UCT that is running long executive certification programs in collaboration with EICT Academy, IITK, IITR and IITG in multiple domains.

2.4 Objectives of this Internship program

The objective for this internship program was to:

- get practical experience of working in the industry.
- to solve real world problems.
- to have improved job prospects.
- to have Improved understanding of our field and its applications.
- to have Personal growth like better communication and problem solving.

2.5 Reference

[1] Node.js & Express.js Developer Documentation
[2] Stripe API Documentation for Secure Checkout and Webhooks
[3] Render & TiDB Cloud Deployment Guidelines

2.6 Glossary

Terms	Acronym
Application Programming Interface	API
Simple Mail Transfer Protocol	SMTP
User Interface / User Experience	UI/UX
Software Development Life Cycle	SDLC
Quality Assurance	QA

3 Problem Statement

Service-based ecosystems often struggle with fragmented communication, unverified transactions, and disorganized scheduling between merchants and customers. Without a centralized hub, service providers face challenges in tracking incoming requests, managing payments securely, and dispatching timely updates to their clients. Furthermore, relying on local testing environments delays the transition to a globally accessible platform.

The objective was to engineer a unified Multi-Client Service Platform. The system required distinct, role-based interfaces for merchants to manage their offerings and for customers to browse, book, and pay for services securely. A critical requirement was moving the architecture to the cloud—hosting the application on Render and the database on TiDB cloud—while seamlessly integrating Stripe for payment processing and an automated email system for reliable order confirmations.

4 Existing and Proposed solution

Existing Solutions:

Many service providers rely on disconnected tools—using one platform for booking, another for manual invoicing, and a third for email communication. These systems are often hosted on fragmented legacy servers, leading to high administrative overhead, database synchronization issues, and a poor user experience.

Proposed Solution:

A centralized, full-stack web platform driven by a robust Node.js backend (server.js). The database is a cloud-native MySQL instance hosted on TiDB cloud and managed via MySQL Workbench. The platform utilizes Stripe API for a unified checkout flow and secure payment processing. Automated backend triggers dispatch dynamic email templates upon registration, successful payment, and order confirmation. The entire system is hosted live using Render for continuous deployment.

Value Addition:

The primary value addition is the transition to a modern, scalable cloud infrastructure. By utilizing Render and TiDB cloud, the platform guarantees high availability. Processing asynchronous Stripe webhooks via the Node.js server updates the remote MySQL database in real-time, eliminating manual intervention. The role-based redirection instantly updates both the customer's order history and the merchant's dashboard simultaneously upon a successful Stripe transaction.

4.1 Code submission (Github link)

https://github.com/tanushreepsahoo2072-debug/upskillcampus/blob/main/MultiClientWebsiteService.zip

4.2 Report submission (Github link)

https://github.com/tanushreepsahoo2072-debug/upskillcampus/blob/main/MultiClientWebsiteService_Tanushree_Priyadarshini_Sahoo_USC_UCT.pdf

4.3 Live Project Hosted On Render (website link)

https://multi-client-service.onrender.com/

5 Proposed Design/ Model

5.1 High Level Diagram (if applicable)

Figure 1: HIGH LEVEL DIAGRAM OF THE SYSTEM

1. **Client Tier:** Dynamic web interfaces tailored for Customers (service browsing/checkout) and Merchants (service management/order tracking).

2. **Application/Logic Tier:** Node.js backend (server.js) managing routes, middleware (node modules), and session management. Secure credentials (Stripe keys, TiDB cloud) are obfuscated via environment variables. Deployed on Render.

3. **Integration Tier:** Bi-directional communication with third-party APIs (Stripe API for transaction tokenization and webhook events; Email API/SMTP for notifications).

4. **Data Tier:** Relational MySQL database hosted on TiDB Cloud, structured for high-speed queries, and managed/configured locally using MySQL Workbench.

5.2 Low Level Diagram (if applicable)

The MySQL database schema on TiDB cloud is structured to maintain strict relational integrity. Key tables include:

- **Users**: Stores encrypted credentials and defines access roles (Customer vs. Merchant).

- **Orders**: Tracks active and past service requests, linking merchant IDs to customer IDs.

- **Payments**: Logs Stripe transaction IDs, calculates merchant pricing tables, and records real-time payment status (Pending, Successful, Failed) triggered by Stripe webhooks.

- **Email_Logs**: Audits the delivery status of transactional emails to monitor system reliability.

5.3 Interfaces (if applicable)

- **Customer Interface:** Designed for a frictionless checkout experience. It features a secure Stripe payment element integration, downloadable payment receipts, and a clean overview of active and historical orders.

- **Merchant Dashboard:** A complex, data-rich interface utilizing CSS Grid/Flexbox for responsiveness. It enables service providers to monitor incoming requests, view Stripe payment statuses, and manage their service catalogs in real-time.

6 Performance Test

Performance testing was heavily focused on the system's live cloud environment, evaluating the latency between Render, TiDB, and Stripe webhooks.

6.1 Test Plan/ Test Cases

1. **Cloud Database Latency:** Measure read/write speeds connecting the Render-hosted Node.js server (server.js) to the TiDB cloud MySQL database.

2. **Stripe Webhook Handling:** Simulate delayed, failed, and successful API responses from Stripe to ensure the remote database updates correctly without hanging the UI.

3. **Email Deliverability:** Verify that dynamic email templates are triggered by the Node server correctly and bypass spam filters.

4. **Live Deployment Integrity:** Ensure node modules are properly installed during the Render build phase and environment variables are successfully injected into the live build.

6.2 Test Procedure

End-to-end testing was conducted on the live URL provided by Render. MySQL Workbench was actively monitored to verify data insertion in real-time. Stripe test cards were used to trigger successful and failed transactions, evaluating the server's webhook parsing logic and error-handling capabilities.

6.3 Performance Outcome

The Render backend successfully managed asynchronous Stripe payment states, updating the TiDB cloud MySQL Payments table accurately with minimal latency. The Node.js automated email system achieved a 100% trigger rate upon successful transactions. The application scaled successfully on Render, with all dependencies in node_modules functioning flawlessly in the production environment.

7 My learnings

This project was a deep dive into modern JavaScript backend development and cloud deployment. I transitioned from local testing environments to configuring live cloud databases using TiDB cloud and MySQL Workbench. I gained a thorough understanding of initializing a Node.js server (server.js), managing dependencies via NPM, and deploying full-stack applications using Render.
Integrating Stripe taught me the complexities of secure tokenization, managing webhook events, and reflecting real-time database changes asynchronously. I learned best practices for cloud security, particularly the critical importance of utilizing environment variables to protect API keys in a production environment.

8 Future work scope

Due to time constraints, the following features have been identified for future implementation:

1. **In-App Messaging:** Developing a direct messaging system (using Socket.io on the Node.js backend) between customers and merchants to discuss service specifics prior to booking.

2. **Advanced Analytics Dashboard:** Integrating data visualization tools so merchants can track their revenue trends, most popular services, and Stripe payout schedules over time.

3. **Multi-Currency Support:** Expanding the Stripe gateway logic to handle dynamic currency conversion for international clients.